

Assignment 2: Coding Basics

Julia Kirkland

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
seq1 <- seq (from=1,to=55,by=5) #used sequence function  
seq1
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2.  
mean(seq1) #used built-in function to determine mean
```

```
## [1] 26
```

```
median (seq1) #used built-in function to determine median
```

```
## [1] 26
```

```

# in the console, I found that the mean=26
# in the console, I found that the median=26

#3.
mean(seq1)>median(seq1) #I asked R a logical test in order to compare the median and mean

## [1] FALSE

# [1] FALSE

```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```

#a) student names
vector1 <- c("John", "Billy", "Hopper", "Star") # Vector type: Character
vector1

```

```
## [1] "John" "Billy" "Hopper" "Star"
```

```

#b) test scores in %
vector2 <- c(79, 81, 83, 94) # Vector type: Numerical
vector2

```

```
## [1] 79 81 83 94
```

```

#c) Scholarship or not
vector3 <- c(TRUE, FALSE, TRUE, FALSE) # Vector type: Logical
vector3

```

```
## [1] TRUE FALSE TRUE FALSE
```

```

create_df <- data.frame(
  "Student"=vector1,
  "Scores"=vector2,
  "Scholarship"=vector3)

create_df

```

```

##   Student Scores Scholarship
## 1   John     79         TRUE
## 2  Billy     81         FALSE
## 3 Hopper     83         TRUE
## 4   Star     94         FALSE

```

```
student_df<-create_df
student_df
```

```
##   Student Scores Scholarship
## 1   John      79         TRUE
## 2  Billy      81        FALSE
## 3 Hopper      83         TRUE
## 4   Star      94        FALSE
```

```
colnames(student_df)<-c("Student Name", "Test Score in %", "Scholarship Status")
student_df
```

```
##   Student Name Test Score in % Scholarship Status
## 1      John      79         TRUE
## 2     Billy      81        FALSE
## 3     Hopper      83         TRUE
## 4       Star      94        FALSE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A data frame can include all different types of data, as seen above. It can have numerical, logical and character data. It also is used to create spreadsheets or CSV files. A matrix only has numerical data and it is a way to do math using R.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
pass_check<-function(score)
{
  if (score>50) print ("pass")
  else {print ("fail")}
}
```

#11. Create a function using ifelse()

```
pass_check_ifelse<-function(score)
{ifelse(score>50, "pass", "fail")}
```

#12a. Run the first function with the value 52.5

```
pass_check(52.5)
```

```
## [1] "pass"
```

```
#12b. Run the second function with the value 52.5  
pass_check_ifelse(52.5)
```

```
## [1] "pass"
```

```
#13a. Run the first function with the vector of test scores  
# pass_check(vector2)
```

```
#13b. Run the second function with the vector of test scores  
pass_check_ifelse(vector2)
```

```
## [1] "pass" "pass" "pass" "pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: `ifelse` worked. This seems to be because you can use vector data in this function. `if...else` only works for single function values.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)